

ECHO: AI-POWERED PERSONAL MEMORY ASSISTANT

A PROJECT REPORT

Submitted by

2116240701503 - SHREENIDHI T

in partial fulfilment of the course

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution)
THANDALAM, CHENNAI – 602 105

October2026



ECHO: AI-POWERED PERSONAL MEMORY ASSISTANT

A PROJECT REPORT

Submitted by

2116240701503 - SHREENIDHI T

in partial fulfilment of the course

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution)
THANDALAM, CHENNAI – 602 105

October2026

BONAFIDE CERTIFICATE

Certified that this project report entitled **"ECHO: AI-POWERED PERSONAL MEMORY ASSISTANT"** is the bonafide work of **Shreenidhi T (240701503)** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____.

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	8
3	PROPOSED SYSTEM	9
4	MODULES DESCRIPTION	12
5	IMPLEMENTATION AND RESULTS	14
6	CONCLUSION AND FUTURE WORK	18
7	REFERENCES	19

ABSTRACT

ECHO is an AI-powered personal memory assistant designed to help users organize important information found in everyday documents such as receipts, warranty cards, medicine labels, and subscription notices. The system aims to simplify information management by combining Optical Character Recognition (OCR), machine-learning-based text classification, database storage, and rule-based reminder logic. The application is developed using Python and Flask, with HTML, CSS, and JavaScript for the user interface and SQLite for data storage. The machine-learning pipeline uses Term Frequency–Inverse Document Frequency (TF-IDF) to convert text into numerical features and Logistic Regression to classify the text into predefined categories such as food, warranty, receipt, document, subscription, medicine, and other. The available project log reports 378 training samples and 98.7% cross-validation accuracy for the classifier.

This result applies only to the classification component and does not represent the accuracy of the complete system. During a recorded test, the application started successfully, but the Tesseract OCR engine was not installed or configured. Therefore, the complete image-scanning workflow requires further configuration and testing. This report describes the background, objectives, literature review, design-thinking process, system architecture, implementation, preliminary testing, limitations, and future enhancements of ECHO.

CHAPTER 1: INTRODUCTION

1.1 Background

In everyday life, people receive various documents containing information that may be required later. These include purchase receipts, warranty cards, medicine labels, subscription notices, and other important records. Such documents may contain purchase details, expiry dates, warranty periods, and renewal information. When these details are scattered across physical documents and digital images, retrieving them at the right time can become difficult.

ECHO is an AI-powered personal memory assistant developed to help users capture, organize, store, and retrieve important information from everyday documents. By integrating OCR, machine learning, database management, and rule-based logic, the system aims to provide a convenient platform for managing personal information.

The application is designed as a web-based system that allows users to interact with its features through a browser. It combines text extraction, automatic categorization, information storage, and reminder management within a single application.

1.2 Problem Focus

The primary focus of ECHO is personal information organization. Instead of requiring users to manually organize every document, the system aims to use OCR to extract text and machine learning to categorize the extracted information.

The project separates text recognition, classification, data storage, and reminder logic into different components. This modular structure makes the application easier to understand, maintain, and improve.

1.3 Modules in the System

The ECHO system consists of the following modules:

1. Image Scanning and OCR Module:

This module accepts document images and attempts to extract readable text using the Tesseract OCR engine when it is installed and configured correctly.

2. Text Classification Module:

The extracted text is processed using TF-IDF to generate numerical features. Logistic Regression then predicts the category of the text.

3. Memory Management Module:

This module manages stored information and allows users to browse their saved memories through the web interface.

4. Reminder Management Module:

The reminder component applies predefined rules to available date information and identifies relevant reminders for display.

5. Question Interface:

The application provides an interface through which users can submit questions to the assistant logic implemented in the project.

6. Dashboard and Settings Module:

The dashboard provides access to the available application features, while the settings page provides access to the application's settings interface.

1.4 Aim and Objectives

The main aim of ECHO is to develop a web-based personal memory assistant that helps users organize important information from everyday documents using OCR, machine learning, and reminder logic.

The objectives are:

- To provide a simple interface for uploading document images.
- To extract readable text from images using OCR when configured.
- To categorize extracted text using a supervised machine-learning model.
- To store memory records in a database.
- To apply rule-based logic for date-related reminders.
- To provide a centralized interface for browsing memories and reminders.
- To evaluate the application and identify opportunities for improvement.

1.5 Scope

ECHO is designed as a lightweight web application for local demonstration and academic evaluation. It focuses on organizing information from personal documents and making stored details easier to retrieve.

The current prototype uses predefined text categories and a browser-based interface. Its effectiveness depends on image quality, OCR configuration, the quality of training data, and the correctness of date information.

The application is intended to support personal organization. It should not be treated as the sole source for medical decisions, legal deadlines, or important financial obligations.

CHAPTER 2: LITERATURE REVIEW

The development of ECHO – AI-Powered Personal Memory Assistant involves technologies such as Optical Character Recognition (OCR), machine learning, database management, and web development. These technologies work together to extract, classify, store, and retrieve information from everyday documents.

2.1 Optical Character Recognition (OCR)

OCR is a technology used to convert text from images and scanned documents into machine-readable text. ECHO uses Tesseract OCR to extract information from receipts, medicine labels, warranty cards, and subscription notices. The quality of the extracted text depends on the clarity of the input image.

2.2 TF-IDF and Logistic Regression

TF-IDF converts text into numerical features that can be processed by machine learning algorithms. Logistic Regression is a supervised learning algorithm used to classify text into predefined categories. ECHO combines these techniques to categorize extracted information, such as receipts, warranties, medicines, and subscriptions.

2.3 Rule-Based Reminder System

Rule-based reasoning uses predefined conditions to make decisions. ECHO uses this approach to process relevant dates and generate reminders for important records, helping users keep track of upcoming events.

2.4 Database and Web Technologies

SQLite is used to store and retrieve personal records, while Flask connects the application's backend components. HTML, CSS, and JavaScript are used to create the user interface and support user interaction.

2.5 Summary

ECHO integrates OCR, TF-IDF, Logistic Regression, rule-based reasoning, SQLite, and Flask into a single application. This combination helps users organize personal information, reduce manual record management, and access important details more conveniently.

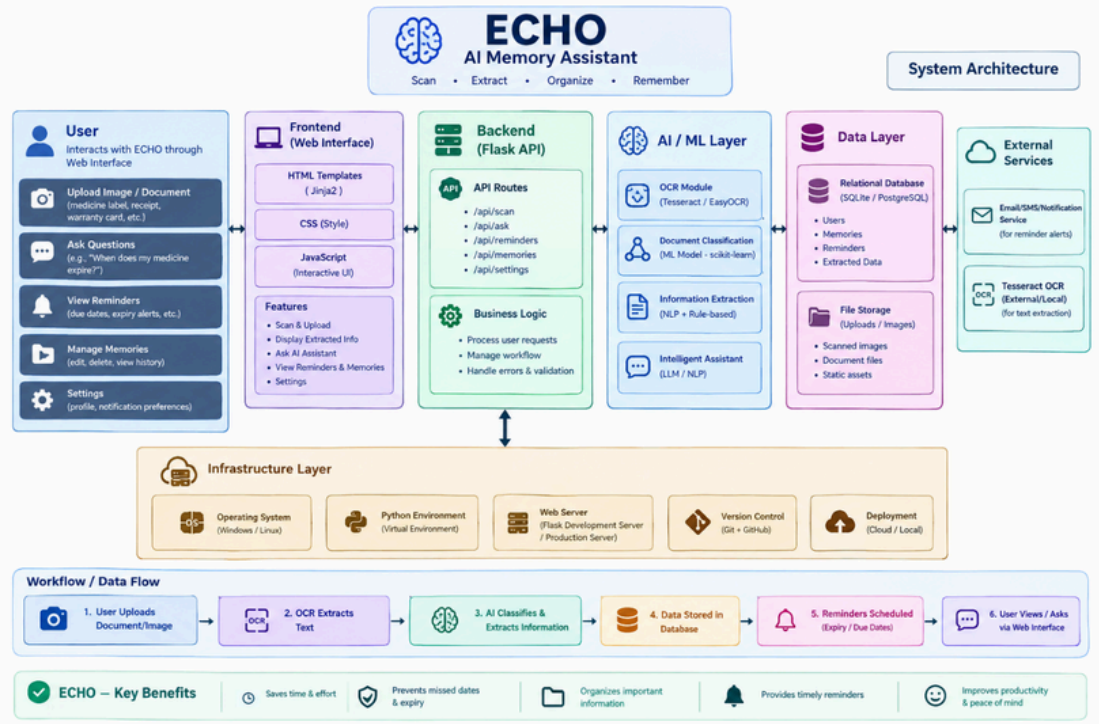
CHAPTER 3: PROPOSED SYSTEM

3.1 System Overview

ECHO – AI-Powered Personal Memory Assistant is a web-based application designed to help users organize and manage important personal information from everyday documents. It uses Optical Character Recognition (OCR) to extract text from uploaded images, machine learning to classify the extracted information, and a database to store records for future access.

The system uses TF-IDF for text feature extraction and Logistic Regression for document classification. Rule-based logic is used to manage relevant dates and reminders. Flask connects the backend components, while HTML, CSS, and JavaScript provide the user interface. The proposed solution reduces the effort required to organize physical documents and helps users access important information conveniently.

3.2 System Architecture



3.3 State Representation

State representation describes the current condition of the system during document processing. In ECHO, a state can be represented using the document's processing status.

- Initial State: The user opens the application and uploads a document.
- OCR State: Text is extracted from the uploaded image.
- Classification State: Extracted text is converted into numerical features and classified.
- Storage State: The processed information is saved in the SQLite database.
- Reminder State: Relevant dates are handled using predefined rules.
- Final State: The information is stored and available for retrieval.

3.4 Algorithms and Techniques Used

1. Optical Character Recognition (OCR)

Tesseract OCR extracts text from uploaded document images. The extracted text is passed to the text-processing module.

2. TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF converts the extracted text into numerical features. These features help the machine learning model identify patterns in the document content.

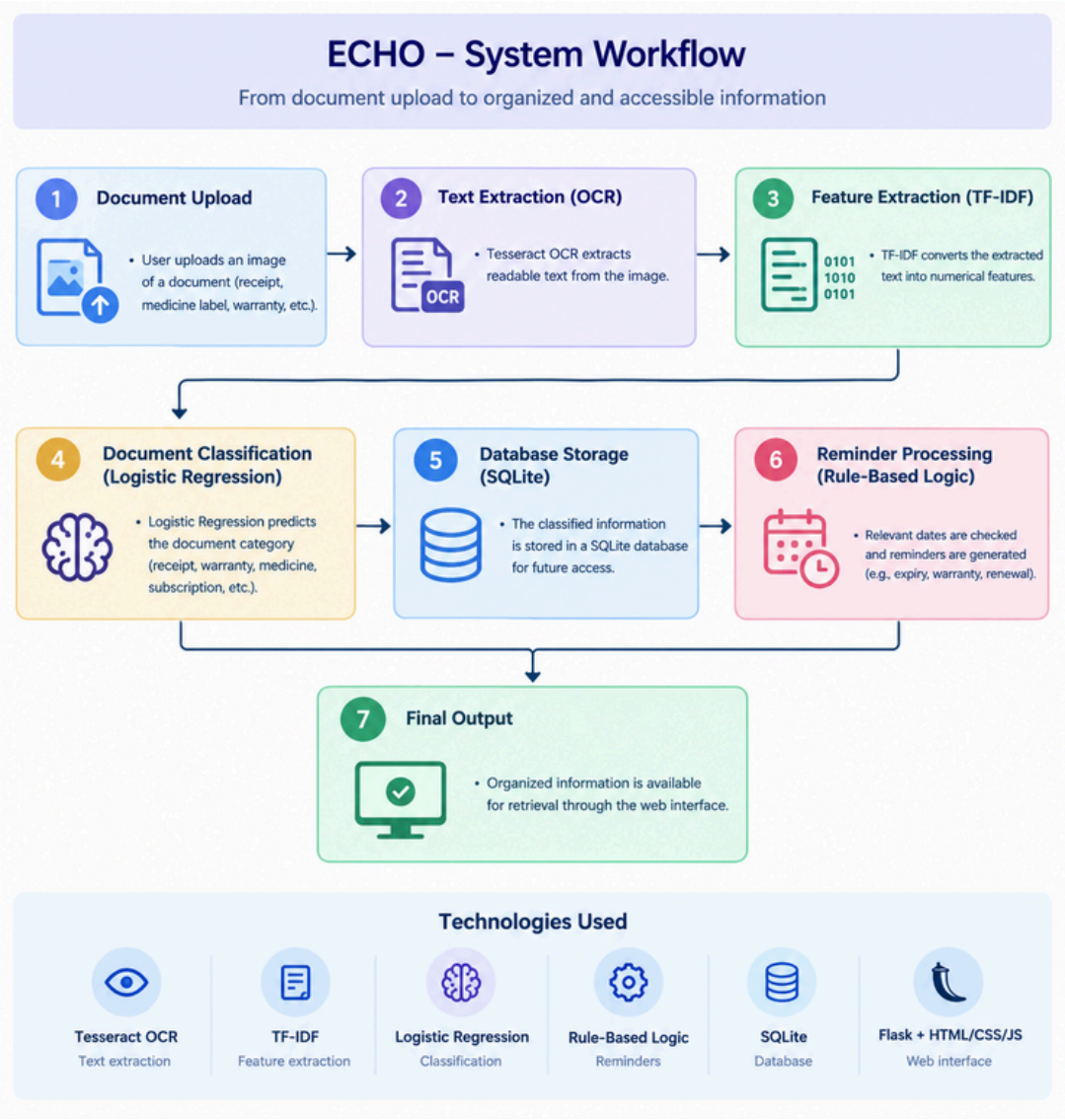
3. Logistic Regression

Logistic Regression is a supervised machine learning algorithm used to classify text into predefined categories. ECHO uses the trained classifier to categorize documents based on their extracted text.

4. Rule-Based Reasoning

Rule-based logic checks predefined conditions related to dates and reminders. It helps the system identify relevant dates and handle reminder-related actions.

3.5 Proposed System Workflow



3.6 Functional Modules

The proposed system consists of four main functional modules:

- Document Management: Supports uploading and managing personal documents.
- Text Processing and Classification: Extracts text and categorizes information using OCR, TF-IDF, and Logistic Regression.
- Information Storage and Retrieval: Uses SQLite to maintain records and retrieve stored information.
- Reminder Management: Uses rule-based logic to handle relevant dates and reminder-related tasks.

CHAPTER 4: MODULES DESCRIPTION

4.1 User Interface and Document Upload Module

This module provides a user-friendly interface for interacting with ECHO. It allows users to upload images of everyday documents, such as receipts, warranty cards, medicine labels, and subscription notices. HTML, CSS, and JavaScript are used to create the interface and support user interactions. The uploaded documents serve as input for further processing.

4.2 Text Extraction Module

The text extraction module uses Optical Character Recognition (OCR) through Tesseract to convert text present in uploaded images into machine-readable text. The extracted information is passed to the text processing module. The quality of extraction depends on the clarity and quality of the uploaded images.

4.3 Text Processing and Classification Module

This module combines text feature extraction and document classification. TF-IDF converts the extracted text into numerical features, which are then processed by the Logistic Regression model. The model predicts the appropriate category, such as receipts, warranties, medicines, subscriptions, documents, or other records. This helps organize personal information systematically.

4.4 Database Management and Information Retrieval Module

This module uses SQLite to store and manage processed personal records. It enables the application to retrieve previously stored information through the user interface. By maintaining organized digital records, the module reduces the effort required to locate important information manually.

4.5 Reminder Management Module

The reminder management module uses rule-based logic to process relevant dates and handle reminder-related actions. It helps users keep track of important dates associated with stored information, such as warranty periods and subscription renewals, depending on the available date information and the rules implemented in the application.

4.6 Backend Integration Module

The backend integration module uses Flask, a Python web framework, to connect the user interface with the application's processing components. It coordinates document processing, classification, database operations, and reminder-related functionality. This integration enables the different modules to work together as a single personal information management system.

CHAPTER 5: IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

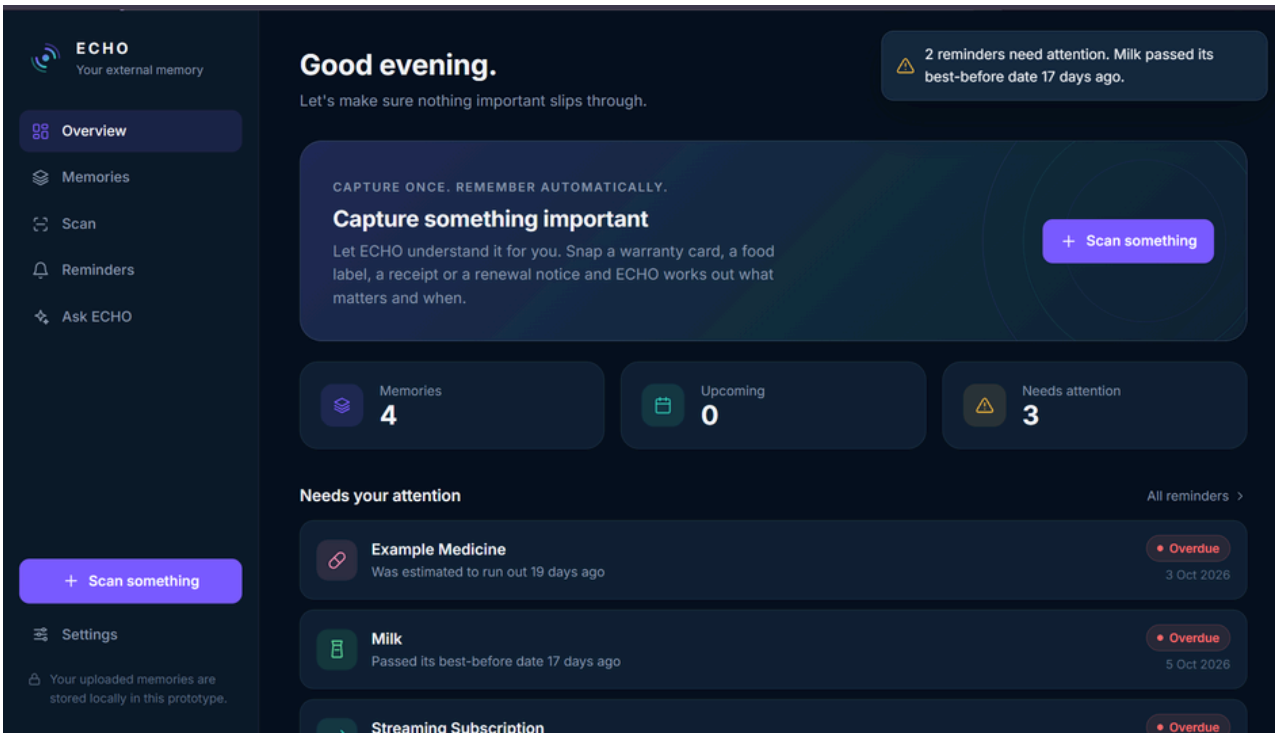
ECHO – AI-Powered Personal Memory Assistant was developed as a web-based application using Python 3.12, Flask, HTML, CSS, JavaScript, and SQLite. The machine learning component uses TF-IDF for text feature extraction and Logistic Regression for document classification. Tesseract OCR is used to extract text from uploaded images, while rule-based logic handles relevant dates and reminders. The application was developed and tested in a local development environment using Visual Studio Code.

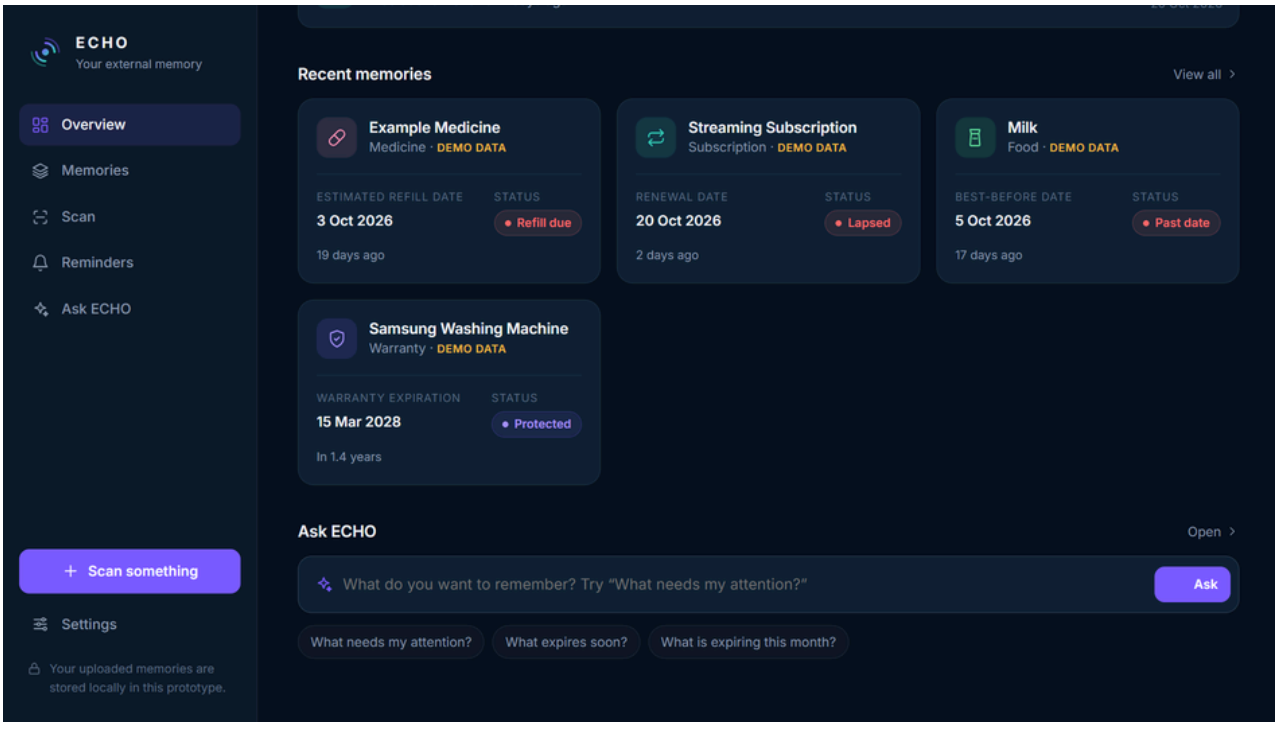
5.2 Test Plans

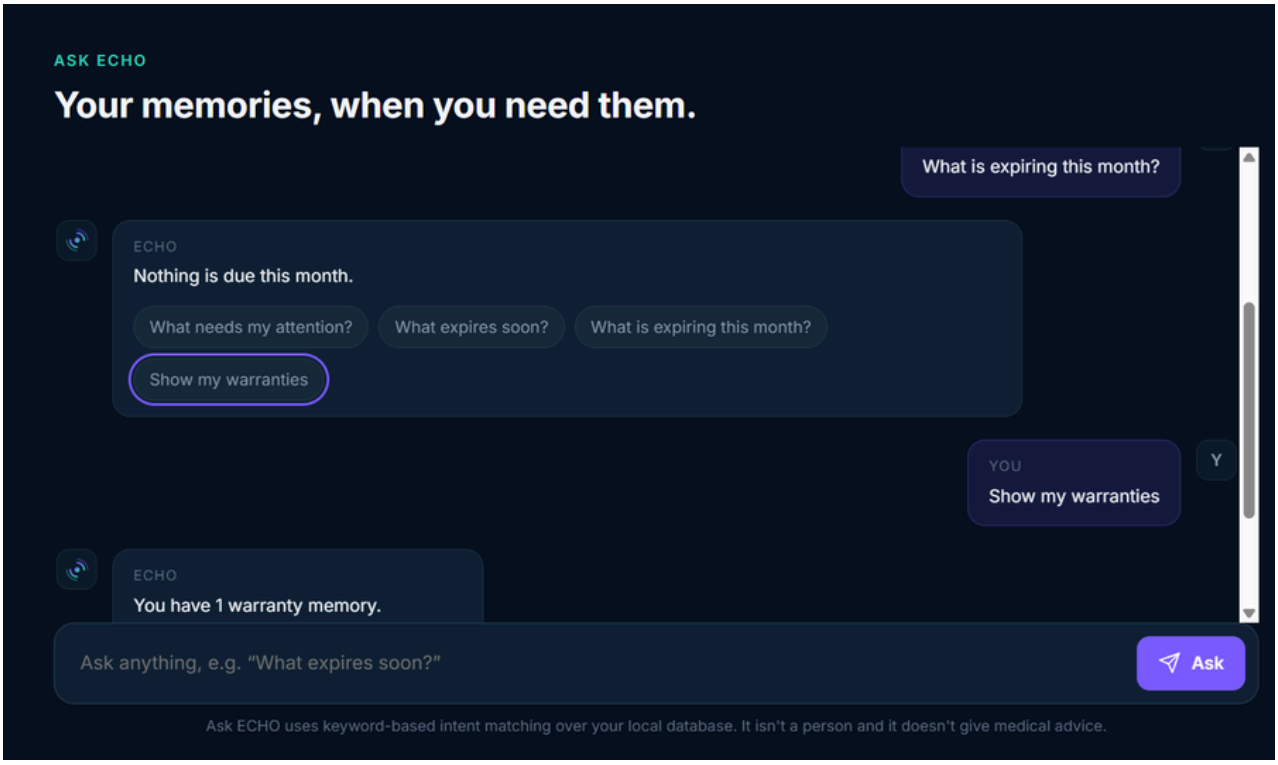
Test Case / Feature	Test Steps	Expected Outcome
Launch the application	1. Open the application in a browser. 2. Check if the home page loads.	Home page loads successfully with all required elements.
Upload a document image	1. Click on the upload option. 2. Select and upload a valid image (e.g., receipt, warranty card).	Image is accepted and displayed for processing.
Extract text using OCR	1. Upload a document image. 2. Trigger the OCR process.	Readable text is extracted from the image.
Classify extracted text	1. Provide extracted text as input. 2. Run the classification model.	Appropriate document category is predicted (e.g., receipt, warranty, medicine, etc.).
Store information	1. Process a document through the complete flow. 2. Check the database.	Record is saved in SQLite with correct details.
Process relevant dates	1. Upload a document with a date (e.g.,warranty or subscription). 2. Check reminder logic.	Reminder is set based on the relevant date and rule.
Retrieve stored records	1. Go to the records section. 2. View previously saved information.	Stored records are displayed correctly in the application.

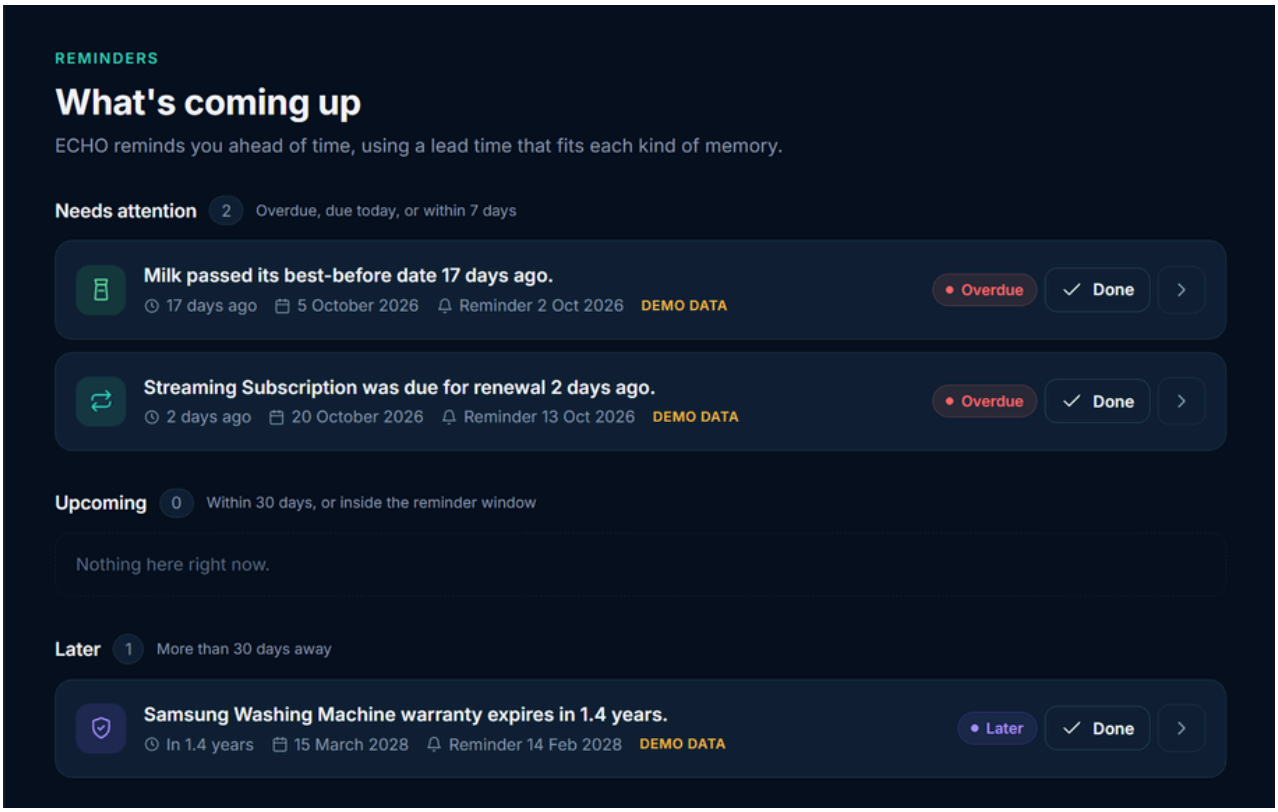
5.3 Results

The Flask application successfully started and served its web pages and static resources. The Logistic Regression classifier, trained using TF-IDF features, achieved 98.7% cross-validation accuracy on the available training dataset. This indicates strong classification performance on the evaluated data.






ASK ECHO
Your memories, when you need them.
What is expiring this month?
ECHO
Nothing is due this month.
What needs my attention?
What expires soon?
What is expiring this month?
Show my warranties
YOU
Show my warranties
ECHO
You have 1 warranty memory.
Ask anything, e.g. "What expires soon?"
Ask
Ask ECHO uses keyword-based intent matching over your local database. It isn't a person and it doesn't give medical advice.


REMINDERS
What's coming up
ECHO reminds you ahead of time, using a lead time that fits each kind of memory.
Needs attention
2
Overdue, due today, or within 7 days
Milk passed its best-before date 17 days ago.
17 days ago
5 October 2026
Reminder 2 Oct 2026
DEMO DATA
Overdue
Done
Streaming Subscription was due for renewal 2 days ago.
2 days ago
20 October 2026
Reminder 13 Oct 2026
DEMO DATA
Overdue
Done
Upcoming
0
Within 30 days, or inside the reminder window
Nothing here right now.
Later
1
More than 30 days away
Samsung Washing Machine warranty expires in 1.4 years.
In 1.4 years
15 March 2028
Reminder 14 Feb 2028
DEMO DATA
Later
Done



5.4 Evaluation Criteria

The system is evaluated using the following criteria:

- **Functional correctness:** Whether the application and its individual modules perform their intended tasks.
- **Classification performance:** How accurately the trained model categorizes text, measured using cross-validation accuracy.
- **Text extraction quality:** Whether OCR extracts readable and relevant text from uploaded images.
- **Data storage and retrieval:** Whether records can be stored and accessed correctly using SQLite.
- **Reminder functionality:** Whether predefined rules process relevant dates as intended.
- **Usability:** Whether users can upload documents and access organized information conveniently.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

ECHO – AI-Powered Personal Memory Assistant is a web-based application designed to simplify the process of organizing and managing personal information from everyday documents. The system integrates Optical Character Recognition (OCR), TF-IDF, Logistic Regression, rule-based reasoning, SQLite, and Flask to extract, classify, store, and retrieve information.

The project demonstrates how artificial intelligence and web technologies can be combined to reduce the manual effort involved in managing documents such as receipts, warranty cards, medicine labels, and subscription notices. The trained classification model achieved 98.7% cross-validation accuracy on the available dataset. However, the complete document-processing workflow requires further validation because Tesseract OCR was not correctly configured in the local environment during testing.

Overall, ECHO provides a foundation for intelligent personal information management, with scope for improving automation, accessibility, and reliability through further development.

6.2 Future Work

1. Improved OCR Integration: Configure and optimize Tesseract OCR to improve text extraction from different document types and image qualities.

2. Advanced Reminder System: Introduce more flexible reminders for warranty expiry, subscription renewals, and other important dates.

3. Cloud Storage and Backup: Integrate secure cloud storage to help users access their information across multiple devices.

4. Enhanced Classification: Train and evaluate the model using a larger and more diverse dataset to improve its ability to classify different documents.

5. Search and Filtering: Add advanced search and filtering options so users can quickly locate specific records.

6. Mobile-Friendly Interface: Improve the interface for mobile devices and consider developing a dedicated mobile application.

7. Security and Privacy: Implement stronger authentication, access control, and data protection mechanisms to safeguard users' personal information.

CHAPTER 7: REFERENCES

1. Smith, R. (2007). “An Overview of the Tesseract OCR Engine.” Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), pp. 629–633. IEEE Computer Society. https://doi.org/10.1109/ICDAR.2007.4376991.

2. Salton, G. and Buckley, C. (1988). “Term-Weighting Approaches in Automatic Text Retrieval.” Information Processing & Management, 24(5), pp. 513–523. https://doi.org/10.1016/0306-4573(88)90021-0.

3. Pedregosa, F., et al. (2011). “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research, 12, pp. 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html.

4. Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/

5. Pallets Projects. Flask Documentation. Available at: https://flask.palletsprojects.com/

6. SQLite Consortium. SQLite Documentation. Available at: https://www.sqlite.org/docs.html

7. Tesseract OCR. Tesseract User Manual. Available at: https://tesseract-ocr.github.io/tessdoc/

8. Scikit-learn Developers. TfidfVectorizer Documentation. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

9. Scikit-learn Developers. LogisticRegression Documentation. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

10. Madmaze. Pytesseract: Python-Tesseract Library. Available at: https://github.com/madmaze/pytesseract